

Football Club & Player Management

J1.L.P0036 | UML, Algorithms, and Test Evidence

Document field	Value
Student	Nguyen Dinh Chuong (Nguyễn Đình Chương)
Student ID	SE204330
Course	LAB211 - OOP with Java Lab
Release	Portfolio review v1.0 - 21 July 2026
Repository	https://github.com/KiritoMainBro88/SU26-LAB211

Scope intentionally limited to the lecturer's review priorities: final UML, implemented algorithms, test cases, and reproducible evidence.

1. Executive Summary

A Java console application for a football league with fourteen required workflows. It models Clubs and Players separately, enforces referential and shirt-number rules, loads strict UTF-8 CSV data, and applies defensive copies at manager boundaries.

1.1 Verification snapshot

Check	Result	Evidence
Java compatibility compile	PASS	javac targeting Java 8, UTF-8, and -Xlint:all
Executable deep verification	62 passed, 0 failed	labs/lab2/testcases/Lab2DeepVerification.java
Import audit	PASS	0 wildcard, 0 unresolved, 0 definitely unused explicit imports
Report/Audit metadata	ALIGNED	J1.L.P0036, SE204330, 21 July 2026
Required functions traced	14/14	Section 5 traceability matrix

1.2 Evidence interpretation

The runnable deep harness was recompiled and executed during this review: 62 checks passed and none failed. The smaller verification_results.txt file is retained as earlier regression evidence (23 passed), but is not counted in the current total.

2. UML Design

This diagram is regenerated from the final source structure and names only implemented classes, interfaces, and relationships relevant to grading and maintenance.

J1.L.P0036 - Final UML Class Relationships

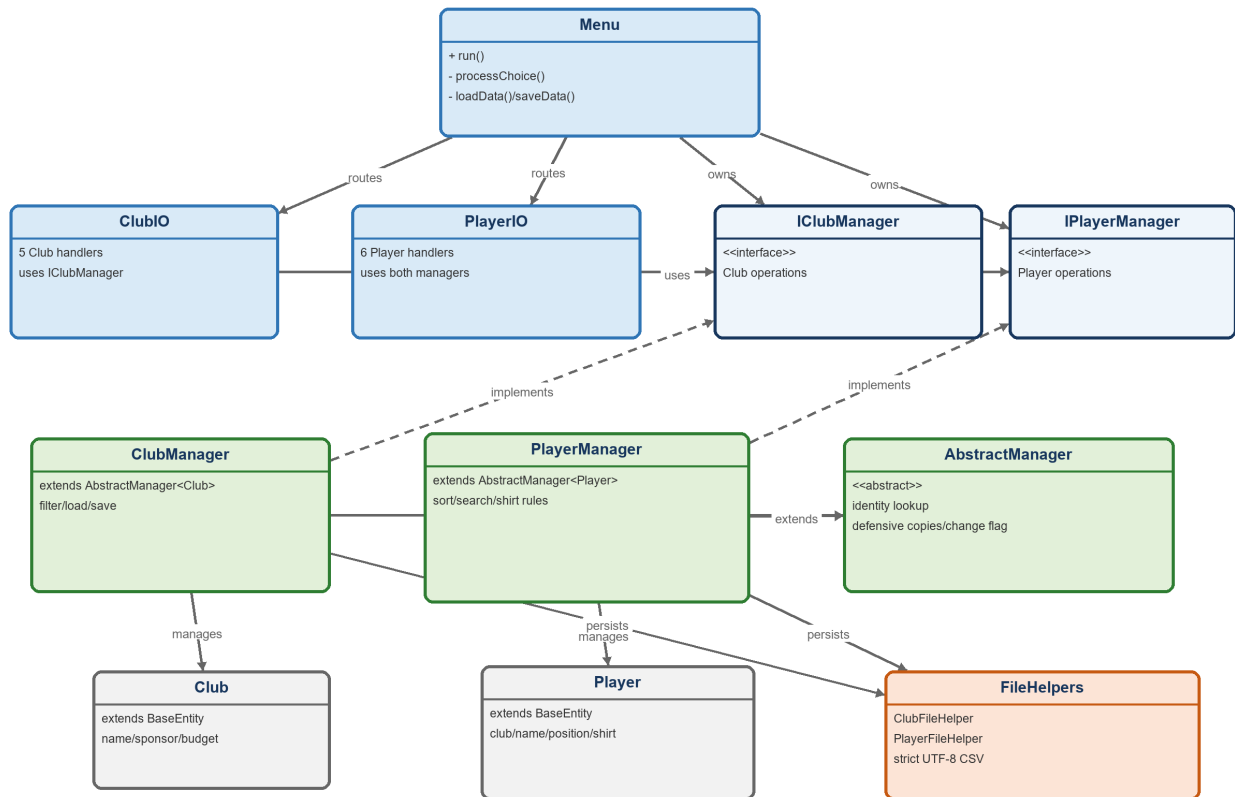


Figure 1. Final class relationships for J1.L.P0036.

2.1 Responsibility matrix

Class / Interface	Final responsibility
Menu	Coordinates fourteen functions and the strict clubs-first reload sequence.
ClubIO / PlayerIO	Own console input/output and delegate business decisions.
IManager<T> / AbstractManager<T>	Provide reusable identity lookup, defensive-copy access, and change tracking.
IClubManager / ClubManager	Implement Club CRUD, budget filtering, validation, load, and save.
IPlayerManager / PlayerManager	Implement Player CRUD, search, filtering, sorting, and shirt uniqueness.
IClubFileHelper / IPlayerFileHelper	Define text persistence boundaries for dependency inversion.
ClubFileHelper / PlayerFileHelper	Parse strict CSV and replace individual targets through temporary files.
BaseEntity / Club / Player / Position	Represent identity, domain data, and the constrained position vocabulary.

2.2 OOP evidence

- Inheritance: Club and Player extend BaseEntity; managers reuse AbstractManager<T>.
- Polymorphism: Menu and IO classes operate through IClubManager and IPlayerManager.
- Encapsulation: managers return entity copies so callers cannot bypass validation.

3. Implemented Algorithms

Pseudocode reflects the final implementation rather than an idealized alternative. Complexity statements account for the list-based structures used by these assignment-sized applications.

Algorithm Flow - Strict Clubs-First Reload

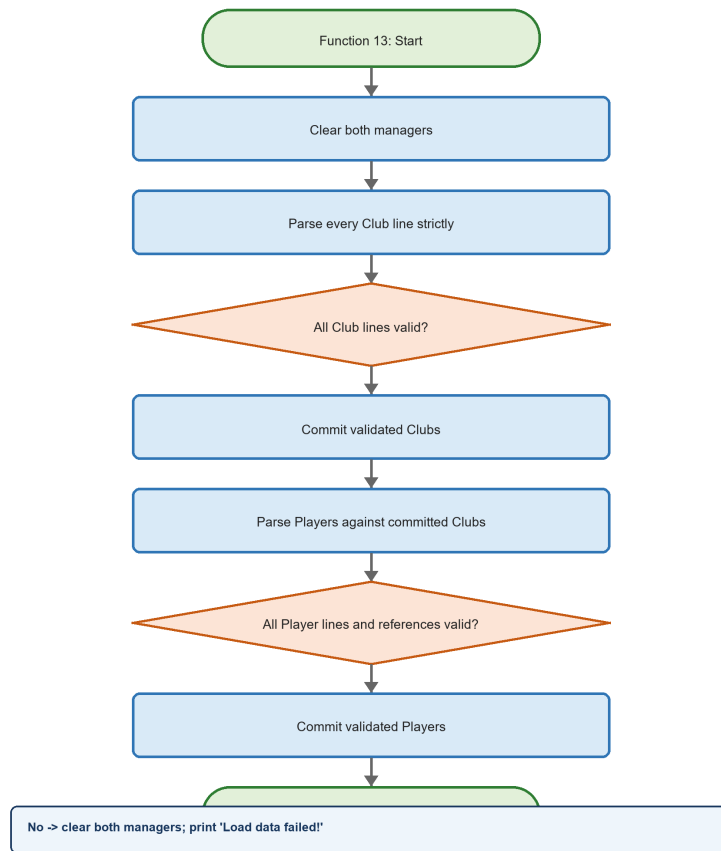


Figure 2. Representative decision flow from the final implementation.

A1 - Strict clubs-first reload

Purpose: Ensure Players never enter memory without a valid referenced Club.

```
01 clear ClubManager and PlayerManager
02 parse every physical club line; reject blank/malformed/duplicate data
03 if club load fails: report failure and remain empty
04 parse players against the validated Club list
05 reject missing club, duplicate ID, invalid position, or duplicate shirt in a Club
06 if player load fails: clear both managers and report failure
07 otherwise commit both validated collections
```

Complexity: $O(c^2 + p(c + p))$ with list-based uniqueness checks; data size is assignment-bounded.

A2 - Add/update Player

Purpose: Maintain identity, reference, position, and per-Club shirt-number invariants.

```
01 validate Pxxxx identity and locate existing Club
02 validate non-empty name and parse constrained Position
03 validate shirt in [1, 99]
04 scan Players for same Club + shirt, ignoring the current ID during update
05 build/modify state only after all validations pass
```

Complexity: $O(c + p)$ time and $O(1)$ additional space.

A3 - Sort Players for Function 6

Purpose: Sort by Club name, then shirt number only when Players share the same Club.

```
01 copy all Players so sorting cannot mutate manager order
02 compare resolved Club names case-insensitively
03 if names differ: return Club-name order
04 if same Club ID: compare shirt number
05 if different Clubs have equal names: use Club ID as deterministic tie-break
```

Complexity: $O(p \log p * c)$ with list-based Club lookup; $O(p)$ result space.

A4 - CSV save with protected replacement

Purpose: Preserve each previous target when validation or writing fails.

```
01 validate complete list and all cross-entity references
02 escape commas, quotes, CR, and LF in text fields
03 write UTF-8 data to a temporary sibling file
04 replace the target atomically when supported; otherwise replace normally
05 delete temporary data on failure and report false
```

Complexity: $O(c + p(c + p))$ validation time and $O(1)$ streaming write space.

4. Test Cases and Executed Evidence

4.1 Execution record

Artifact	Status	Interpretation
Source compilation	PASS	All production and deep-test Java files compile for Java 8.
Deep harness	62/62 PASS	Re-executed 21 July 2026; exit code 0.
Earlier regression record	23/23 PASS	Historical result; not added to the current 153-check claim.
State preservation	PASS	Failure paths assert memory/file content, not only return values.

4.2 Representative test matrix

ID	Scenario	Input / action	Expected result	Actual
L2-T01	Club boundaries	null, NaN/Infinity budget, duplicate ID	Rejected; valid finite Club accepted	PASS
L2-T02	Player constraints	null position, shirt 100, missing Club	Rejected before insertion	PASS
L2-T03	Shirt-number scope	Same number in same and different Clubs	Same-Club duplicate rejected; cross-Club accepted	PASS
L2-T04	Blank update fields	Name, position, and shirt blank	Original fields retained; operation succeeds	PASS
L2-T05	Sort tie-breaks	Equal Club names; same-Club shirts	Club ID tie-break; same Club by shirt	PASS
L2-T06	Strict physical lines	Blank Club line during reload	Reload fails and both managers remain empty	PASS
L2-T07	Quoted CSV round-trip	Comma/quote in Club and Player names	Text preserved exactly	PASS
L2-T08	Defensive results	Mutate get/find/search/filter result	Manager state remains unchanged	PASS
L2-T09	Finite threshold	0, -1, NaN, Infinity	Finite values accepted; non-finite rejected	PASS
L2-T10	Manager persistence APIs	Save and load via both managers	Round-trip succeeds	PASS

4.3 Coverage strategy

- Equivalence partitions: valid, invalid format, duplicate, missing reference, and empty/null inputs.
- Numeric boundaries: lower/upper limits plus NaN and positive infinity for double values.
- State assertions: rejected operations preserve the prior object list or target file.
- Persistence round-trips: valid object/text data reloads with essential values preserved.
- Encapsulation checks: mutations to returned objects/lists cannot alter managed state.

5. Requirement-to-Source Traceability

Every visible assignment function maps to a final handler, implementation method, and executable or directly inspectable verification artifact.

Requirement	Menu / handler	Final implementation	Evidence
1. List Clubs	Menu case 1	ClubIO.listAllClubs	Table output / empty list
2. Add Club	Menu case 2	ClubManager.addClub	L2-T01
3. Search Club	Menu case 3	ClubManager.getClubById	Found/not-found
4. Update Club	Menu case 4	ClubManager.updateClub	Blank/invalid update
5. Budget Filter	Menu case 5	ClubManager.filterByBudget	L2-T09
6. Sorted Players	Menu case 6	PlayerManager.getPlayersSortedByClubName	L2-T05
7. Search Players	Menu case 7	PlayerManager.searchByPartialName	Case-insensitive/null
8. Add Player	Menu case 8	PlayerManager.addPlayer	L2-T02, T03
9. Remove Player	Menu case 9	PlayerManager.removePlayer	Found/not-found
10. Update Player	Menu case 10	PlayerManager.updatePlayer	L2-T03, T04
11. Position Filter	Menu case 11	PlayerManager.filterByPosition	Enum/null
12. Save	Menu case 12	ClubManager/PlayerManager.saveToFile	L2-T07, T10
13. Load	Menu case 13	Menu.loadData	L2-T06
14. Quit	Menu case 14	Menu.quitProgram	Changed-state save path

5.1 Import and source hygiene

Audit item	Result	Basis
Incorrect/unresolved imports	0	Complete production + test compilation succeeded.
Wildcard imports	0	Repository-wide source scan.
Definitely unused explicit imports	0	Imported simple names are referenced in their units.
Generated binaries committed	0	.gitignore excludes build, dist, class, jar, private IDE state, and logs.

5.2 Known limitations and deliberate trade-offs

- Clubs and Players are saved sequentially. Each target is protected, but the two files are not one transaction.
- Sorting resolves Club names through list lookup; a map would improve asymptotic performance for large data sets.
- Strict Function 13 rejects blank physical lines by design, matching the lecturer requirement.

5.3 Reproduction

From the repository root on Windows:

```
PowerShell -ExecutionPolicy Bypass -File .\scripts\verify-all.ps1
```

GitHub Actions invokes the same script on every push and pull request. The workflow pins actions to reviewed commit SHAs and grants read-only repository permission.

Conclusion

The final J1.L.P0036 source implements all 14 required functions, compiles for Java 8, and passes 62 executable deep checks. The UML, algorithms, test evidence, and AI Audit Log use the same identifiers, counts, paths, and verification date.